DS Prácticas

Beatriz Ruz Gómez
Yeray Rodríguez
Pablo De la Torre Roldán

21 de marzo de 2026

Índice

Ejercicio 1 **3**

Ejercicio 2 **5**

Ejercicio 3 **8**

Ejercicio 4 **12**

Ejercicio 1

El ejercicio consiste en programar una simulación de partidas multijugador multihebra que tengan el mismo número inicial N de jugadores (introducido por el usuario). En las partidas competitivas el 20 % de los jugadores hacen ragequit, mientras que en las partidas casuales solamente lo hacen el 10 %. El tiempo de cada partida son 60 segundos y todos los jugadores se salen a la vez (tanto en la partida competitiva como en la casual).

Hemos utilizado 4 archivos .java, los cuales son:

- **FactoriaPartidaYJugador.java**: Archivo que implementa las 3 clases factoría:
  - **FactoriaPartidaYJugador**: Es la interfaz que hace de "factoría abstracta". Declara las funciones crearPartida() y crearJugador().
  - **FactoriaCasual**: Es la clase que crea los productos casuales. Implementa FactoriaPartidaYJugador.
  - **FactoriaCompeti**: Es la clase que crea los productos competitivos. Implementa FactoriaPartidaYJugador.
- **Jugador.java**: Contiene las 3 clases asociadas a jugadores:
  - **Jugador**: Clase abstracta de la que heredan JugadorCasual y JugadorCompeti. Todo jugador tiene guardado su id en la partida como un atributo de instancia.
  - **JugadorCasual**: Clase producto que contiene los jugadores casuales.
  - **JugadorCompeti**: Clase producto que contiene los jugadores competitivos.
- **Partida.java**: Contiene las 3 clases asociadas a partidas:
  - **Partida**: Clase abstracta de la que heredan por PartidaCasual y PartidaCompeti. Toda partida tiene una lista de jugadores como atributo de instancia. Partida implementa el método addJugador() y define el método abandonar().
  - **PartidaCasual**: Clase producto que contiene las partidas casuales. Implementa la clase Runnable para ejecutar el programa en distintas hebras con el método run(). En dicho método se esperan 30 segundos, el 10 % de los jugadores abandonan y finaliza la partida después de otros 30 segundos.
  - **PartidaCompeti**: Clase producto que contiene las partidas competitivas. Implementa la clase Runnable para ejecutar el programa en distintas hebras con el método run(). En dicho método se esperan 30 segundos, el 20 % delos jugadores abandonan y finaliza la partida después de otros 30 segundos.
- **Cliente.java**: Es el main del programa con el cual interactúa el usuario. Se encarga de crear las partidas y jugadores con sus respectivas factorías, añadir los jugadores a las partidas y ejecutarlas en paralelo con hebras. Se han usado objetos de tipo **Thread** para hacer la ejecución en paralelo, aunque las partidas implementan Runnable.

Este es el diagrama UML de nuestras clases:

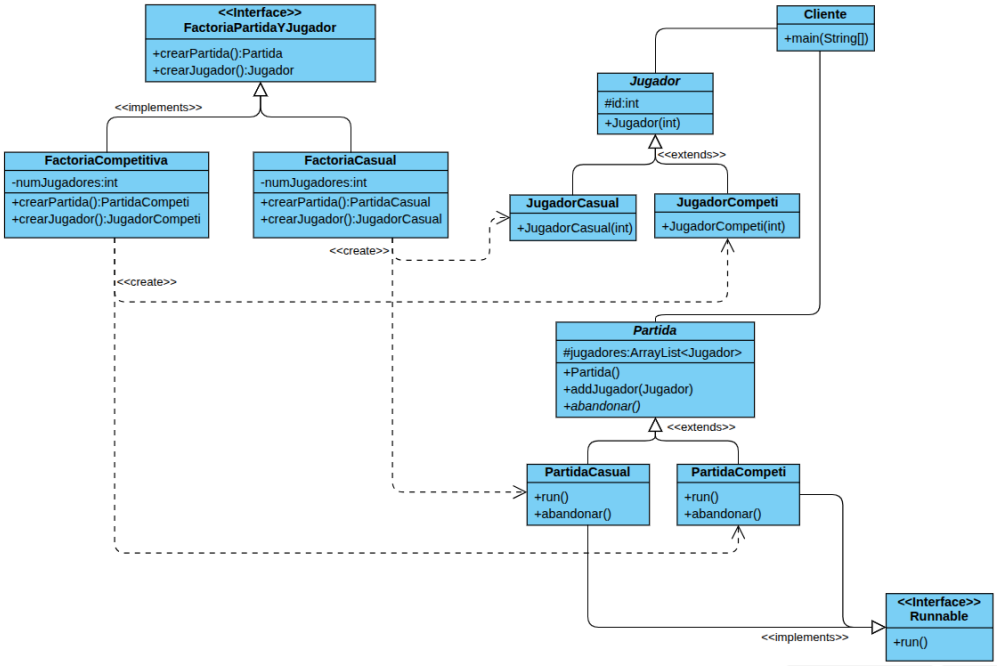

Figura 1: Diagrama UML: Patrón Factoría Abstracta

Para ejecutar el programa se debe hacer "javac *.java && java Cliente.en la terminal, en la carpeta Ejercicio1

Ejercicio 2

En este ejercicio se propone utilizar la API de Hugging Face para, usando distintos modelos, resumir, traducir y generar un análisis de sentimientos de un texto. Todo esto ha de hacerse utilizando el **Patrón Decorador**, lo que nos permite tener un modelo base, en este caso el resumidor, al que le podemos añadir al gusto más funciones, como la de traducir, generar el análisis de sentimientos o ambas a la vez, pudiendo encadenarse.

Hemos necesitado un total de **4 archivos .py** y **1 archivo .json**, los cuales son:

- **LLM.py**: Contiene la interfaz que declara el método **generate_summary** y la función **query_api** para realizar peticiones a la API de Hugging Face.
- **BasicLLM.py**: Es el modelo básico, implementa el método generate_summary para realizar un resumen con el modelo especificado en el JSON.
- **Decorator.py** : Implementa el patrón decorador. Se compone de tres elementos:
  - `Decorator` **(Clase Base):** Funciona como la plantilla estructural para los decoradores concretos.
  - `TranslationDecorator`**:** Obtiene el texto en inglés de la capa previa (resumen generado por el LLM), comprueba que no existan errores, y luego realiza una llamada a la API (`query_api`) para procesarlo, devolviendo el texto traducido.
  - `SentimentDecorator`**:** Obtiene el texto en inglés de la capa previa, lo envía a un modelo multilingüe a través de la API, y devuelve el texto recibido junto con su etiqueta de sentimiento final.
- **Main.py**: Abre el JSON para obtener los datos e instancia objetos de las distintas clases. Llama a los métodos generate_summary para resumir, traducir y valorar con estrellas. Por último, encadena las tres clases para hacer las tres funciones a la vez.

  Es importante destacar que el orden en el que se anidan los decoradores influye. Para que el resultado tenga sentido, hemos elegido el siguiente orden (primero resumen, luego traduccion y finalmente valoración):

  ```
  resultado4 = SentimentDecorator(TranslationDecorator(llm, model_translation,
   token), model_sentiment, token).decorate(texto)
  print(resultado4)
  ```

  Esta decisión se debe a que no tiene sentido hacer un resumen o traduccion sobre una valoracion ya que el resultado es un número de estrellas.
- **Configuración.json**: Permite almacenar y compartir datos entre sistemas o entre las distintas capas de una aplicación web, estructurados de forma ligera y legible.

Peticiones a la API

Al realizar la petición HTTP mediante la librería requests, la API responde enviando un bloque de texto en formato JSON. En nuestro código, la llamada a response.json() se encarga de convertirlo en estructuras de datos nativas de Python (listas y diccionarios). Los distintos tipos que devuelven los modelos son:

- Resumen: Devuelve una lista que contiene un único diccionario. La cadena de texto generada se encuentra asociada a la clave summary_text.
- Traducción: Sigue un patrón similar al anterior, devolviendo una lista con un diccionario, pero en este caso el texto de salida se asocia a la clave translation_text.
- Análisis de Sentimientos: Presenta una estructura más anidada. Devuelve una lista que contiene otra lista, dentro de la cual se encuentra el diccionario con las claves de clasificación y probabilidad. En este ejercicio, se extrajo el valor asociado a la clave label.

Este es el diagrama UML de nuestras clases:

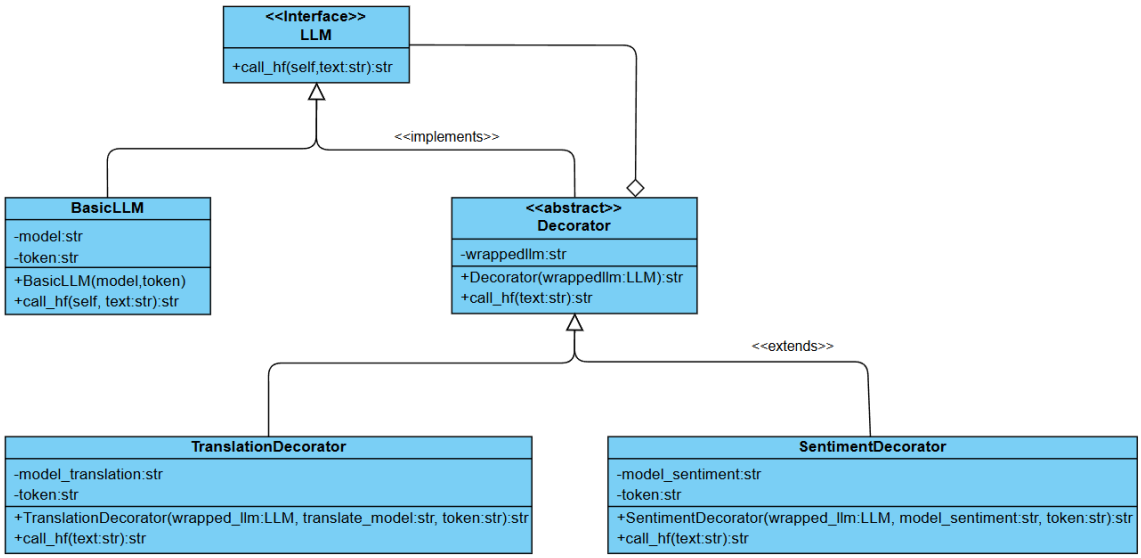

Figura 2: Diagrama UML: Patrón Decorador

Ejemplo de ejecución:

```
yeray@yeray-Zenbook:~/DSPracticas/Practica1/Ejercicio2$ python3 main.py
===========Texto Original===========
In my restless dreams, I see that town Silent Hill You promised me you'd take me there again someday But you never did Well, I'm alone t
here now... In our 'special place'... Waiting for you... to come see me But you never do And so I wait, wrapped in my cocoon of pain and
 loneliness I know I've done a terrible thing to you I wish I could change that but I can't I feel so pathetic and ugly lying here Every
 day I stare up at the cracks in the ceiling and all I can think about is how unfair it is... The doctor came today. He told me I could
go home for a short stay. It's not that I'm getting better. It's just that this may be my last chance... Even so, I'm glad to be coming
home. I've missed you terribly. But I'm afraid, James. I'm afraid you don't really want me to come home. Whenever you come see me, I can
 tell how hard it is on you... I don't know if you hate me or pity me... Or maybe I just disgust you... I'm so sorry. When I first learn
ed that I was going to die, I couldn't accept it. I was so angry I struck out at everyone I loved. Especially you. That's why I understа
nd if you do hate me. But I want you to know. I'll always love you. Even though our life together had to end like this, I still wouldn't
 trade it for the world. Well, this letter has gone on too long, so I'll say goodbye. I told the nurse to give this to you after I'm gon
e. That means that as you read this, I'm already dead. But that's okay, I'm not afraid of it anymore. I just hope the pain will end soon
. So that you remember me for who I was... and not for what the disease made me. I want you to go on. I want you to live. For yourself a
nd for others. Like you did for me. James... You made me happy.

===========LLM Básico===========

Generating summary using model facebook/bart-large-cnn

"I know I've done a terrible thing to you. I wish I could change that but I can't" "I feel so pathetic and ugly lying here Every day I s
tare up at the cracks in the ceiling and all I can think about is how unfair it is... The doctor came today. He told me I could go home
for a short stay. It's just that this may be my last chance... Even so, I'm glad to be coming home. I've missed you terribly"
```

```
===========LLM con Decorator de Traducción===========

Generating summary using model facebook/bart-large-cnn

Traduciendo usando el modelo Helsinki-NLP/opus-mt-en-es

"Sé que te he hecho algo terrible. Ojalá pudiera cambiar eso, pero no puedo" "Me siento tan patética y fea aquí tirada Todos los días mi
ro las grietas en el techo y todo lo que puedo pensar es en lo injusto que es... el doctor vino hoy. Me dijo que podía ir a casa por una
 corta estancia. Es sólo que esta puede ser mi última oportunidad... Aún así, me alegro de volver a casa. Te he echado terriblemente de
menos"

===========LLM con Decorator de Sentimiento===========

Generating summary using model facebook/bart-large-cnn

Generando analisis de sentimientos con el modelo nlptown/bert-base-multilingual-uncased-sentiment

"I know I've done a terrible thing to you. I wish I could change that but I can't" "I feel so pathetic and ugly lying here Every day I s
tare up at the cracks in the ceiling and all I can think about is how unfair it is... The doctor came today. He told me I could go home
for a short stay. It's just that this may be my last chance... Even so, I'm glad to be coming home. I've missed you terribly"
[Sentimiento: 1 star]
```

```
===========LLM con Decorator de Traducción y Sentimiento===========

Generating summary using model facebook/bart-large-cnn

Traduciendo usando el modelo Helsinki-NLP/opus-mt-en-es

Generando analisis de sentimientos con el modelo nlptown/bert-base-multilingual-uncased-sentiment

"Sé que te he hecho algo terrible. Ojalá pudiera cambiar eso, pero no puedo" "Me siento tan patética y fea aquí tirada Todos los días mi
ro las grietas en el techo y todo lo que puedo pensar es en lo injusto que es... el doctor vino hoy. Me dijo que podía ir a casa por una
 corta estancia. Es sólo que esta puede ser mi última oportunidad... Aún así, me alegro de volver a casa. Te he echado terriblemente de
menos"
[Sentimiento: 1 star]
```

Ejercicio 3

El objetivo de este ejercicio consiste en **extraer información estructurada sobre equipos de hockey** desde el sitio web de pruebas *Scrape This Site* (https://www.scrapethissite.com/pages/forms/). El requisito principal es obtener los datos de las primeras 5 páginas y exportarlos a un formato CSV.

Para el objetivo propuesto, en el cual queremos que el usuario sea capaz de elegir en tiempo de ejecución qué estrategia de scrapping desea utilizar de entre las dos propuestas, **hemos optado por implementar el Patrón Strategy** por la aplicación de dos métodos distintos para obtener un mismo resultado. Hemos necesitado un total de **5 archivos .py**, los cuales son:

- **ClaseScrape.py**
- **ClaseContexto.py**
- **ClaseSelenium.py**
- **ClaseBeautifulSoup.py**
- **Main.py**

Este es el diagrama UML de nuestras clases:

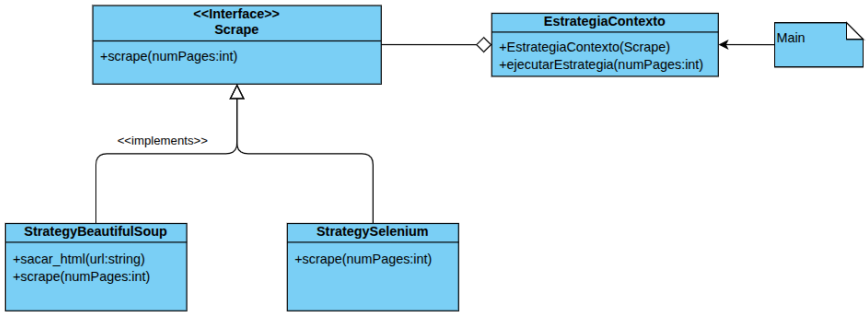

Figura 3: Diagrama UML: Patrón Strategy

Esta estructura de clases nos permite decidir la estrategia gracias a la clase Contexto, que almacena la estrategia elegida por el usuario.

Para la realización de este ejercicio, hemos necesitado utilizar las librerías **BeautifulSoup y Selenium**, lo que ha implicado añadir las siguientes líneas en sus respectivas clases:

- **BeautifulSoup**:

```
from bs4 import BeautifulSoup
from requests import
```

- **Selenium**j:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
```

Hemos definido una interfaz común o clase padre (Scrape), de la cual heredan nuestras dos estrategias concretas:

- **StrategyBeautifulSoup:** Analiza el HTML estático.
- **StrategySelenium:** Automatiza un navegador para extraer los datos renderizados.

Gracias a este patrón, hemos podido dividirnos el trabajo mediante VS Code Live Share y GitHub sin generar conflictos en el código, ya que pudimos editar las estrategias y distintos archivos del proyecto sin afectar a los demás.

BeautifulSoup

Para implementar la solución con el método de BeautifulSoup, utilizamos la librería requests para descargar el código fuente de las cinco primeras páginas. Una vez obtenido el HTML, BeautifulSoup nos permite iterar sobre los datos obtenidos mediante sus etiquetas. En el caso de esta página web, utiliza las etiquetas `<tr>` y `<td>` de la clase team, donde `<tr>` almacena los nombres de los equipos y `<td>` sus demás atributos. Una vez obtenemos los datos que necesitamos, los convertimos a texto sin espacios en blanco innecesarios con text.stripe().

Obtenemos el HTML ordenado inicializando una instancia de la clase de esta manera:

```
BeautifulSoup(html, html.parser)
```

Para sacar las etiquetas y lo que contienen usamos las funciones:

```
find("nombre_etiqueta"), que obtiene una coincidencia de las que tengan esa etiqueta
find_all("nombre_etiqueta"), que obtiene todo lo que tenga esa etiqueta
```

Selenium

En el caso de Selenium, utilizamos la librería de Selenium para automatizar un navegador web (en nuestro caso, Chromium) y simular la navegación por las cinco primeras páginas (O el número de páginas elegido por el usuario que se pasa como parámetro **numPages**):

```
class StrategySelenium(Scrape):
    def scrape(self, numPages):
```

A diferencia de BeautifulSoup, Selenium no solo descarga el texto, sino que renderiza el DOM.

Hemos optado por definir manualmente `driver_location` y `binary_location`, ya que no se encontraba la ruta de los drivers de Chromium, y este no era el navegador por defecto.

Además, utilizamos `driver.implicitly_wait(5)` y `driver.quit()`. La espera permite al navegador que, si no encuentra un elemento, espere hasta 5 segundos antes de lanzar un error, lo que permite pequeños retrasos de red. Por su parte, `driver.quit()` garantiza el cierre completo del navegador al finalizar la ejecución, evitando procesos en segundo plano.

Para la extracción de datos en Selenium usamos la misma lógica que en BeautifulSoup pero con el método equivalente `find_elements` con selectores de tipo `By.TAG_NAME` sobre las etiquetas `<tr>`, `<th>` y `<td>`. Esta elección permite consultar directamente el árbol de elementos, en lugar de parsear texto plano. Asimismo, se mantiene la lógica `if(i == 1)` para capturar los encabezados `<th>` únicamente en la primera página.

Por último, Selenium ya realiza una limpieza automática de espacios en blanco y etiquetas internas mediante la propiedad `.text`, sin necesidad de `.stripe()`, entregándote el dato listo para ser procesado.

Main.py

La lógica del main es bastante sencilla. Primero pedimos al usuario introducir el método de scraping (BeautifulSoup o Selenium) y luego le pedimos que indique el número de páginas a scrappear.

Una vez que sabemos que método de scrapping se va a usar, creamos un objeto EstrategiaContexto al que le pasamos como parámetro del constructor el método a utilizar:

```
if opcion == 1:
    scrapper = EstrategiaContexto(StrategyBeautifulSoup())
if opcion == 2:
    scrapper = EstrategiaContexto(StrategySelenium())
```

Esto nos permite llamar a la función `ejecutar_estrategia(int numPages)` de la misma forma, independientemente del scrapper elegido. Además nos facilitaa el diseño de la función que genera los csv. Ésta comprueba si el atributo estrategia de nuestra variable scrapper es de la clase StrategyBeautifulSoup o StrategySelenium para elegir el nombre del archivo.

```
def generaCSV(resultado, scrapper):
    if isinstance(scrapper.estrategia, StategyBeautifulSoup):
        nombre = "equiposBS.csv
```

```
elif isinstance(scrapper.estrategia, StrategySelenium):
    nombre = equiposSelenium.csv
```

Ejercicio 4

En este ejercicio se pide usar el patrón filtros de intercepción para comprobar que la entrada del usuario cumpla una serie de criterios. El usuario introduce un correo y una contraseña que el programa representa como un Struct (correo, password). El programa luego pasa 2 filtros al correo y 3 a la contraseña. Si no pasa alguno de los filtros el programa notifica al usuario del error.

Hemos usado un total de 11 archivos .rb:

- **StructCredenciales.rb**: El archivo que define el struct de credenciales (correo, password).
- **Main.rb**: Es el archivo con el que se ejecuta el programa y el equivalente al cliente. Se encarga de crear el Target y el FilterManager, añadir los filtros a FilterManager y obtener las credenciales del usuario, además de ejecutar los filtros.
- **FilterManager.rb**: Archivo que implementa la clase FilterManager. Contiene un atributo de tipo FilterChain y permite tanto añadir un filtro al FilterChain como mandar las credenciales al FilterChain para que éste ejecute los filtros.
- **FilterChain.rb**: Archivo que implementa la clase FilterChain. Contiene una lista de filtros y un Target como atributos. Su función es llenar la lista de Filter y ejecutar los filtros.
- **Filter.rb**: Interfaz que define el método que usarán los distintos filtros. Se usa un módulo para simular una interfaz.
- **FilterDomain.rb**: Filtro que implementa a Filter y que se encarga de comprobar que el correo finalice con "@gmail.com.º "@hotmail.com".
- **FilterEmail.rb**: Filtro que implementa a Filter y que se encarga de comprobar que el principio del correo no esté vacío y no contenga varios "@".
- **PwCapsFilter.rb**: Filtro que implementa a Filter y que se encarga de comprobar que la contraseña tenga mayúsculas y minúsculas.
- **PwCharactersFilter.rb**: Filtro que implementa a Filter y que se encarga de comprobar que la contraseña tenga caracteres especiales.
- **PwLengthFilter.rb**: Filtro que implementa a Filter y que se encarga de comprobar que la contraseña tenga al menos 10 caracteres.
- **Target.rb**: Clase objetivo que es interceptada por los filtros.

Este es el diagrama UML de nuestras clases:

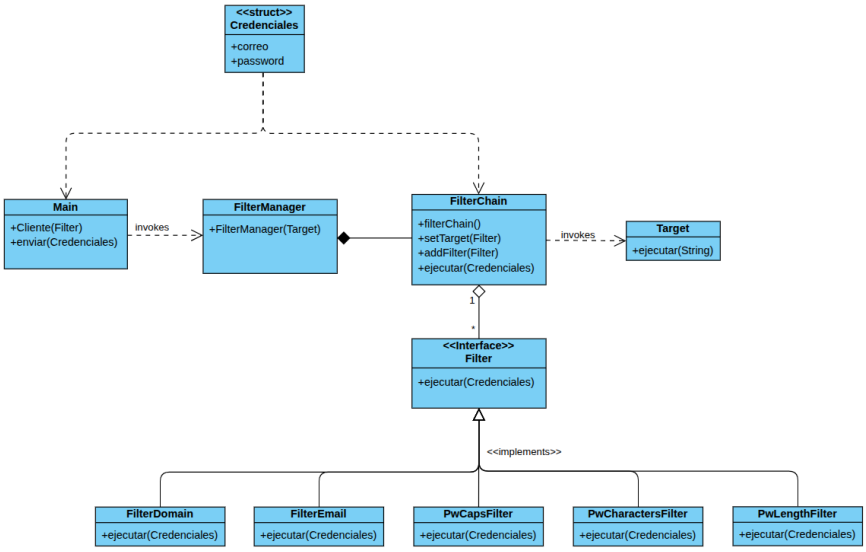

Figura 4: Diagrama UML: Patrón Filtros de intercepción